



UEA

UNIVERSIDAD
ESTATAL AMAZÓNICA





UEA
UNIVERSIDAD
ESTATAL AMAZÓNICA

ESTRUCTURA DE DATOS



SEMANA 13

DESARROLLO DE LA SEMANA 13: DEL LUN. 09 - DOM. 15 DE SEPTIEMBRE/2024

Resultado de aprendizaje: Implementar las operaciones de árboles y grafos, mediante el uso de un lenguaje de programación, para la resolución de problemas que requieran ordenamiento y búsquedas eficientes y la construcción del árbol de expansión mínima

CONTENIDOS

UNIDAD IV

- **Tema 1: Árboles binarios**
 - **Subtema: Árboles binarios de búsqueda: características y propiedades.**



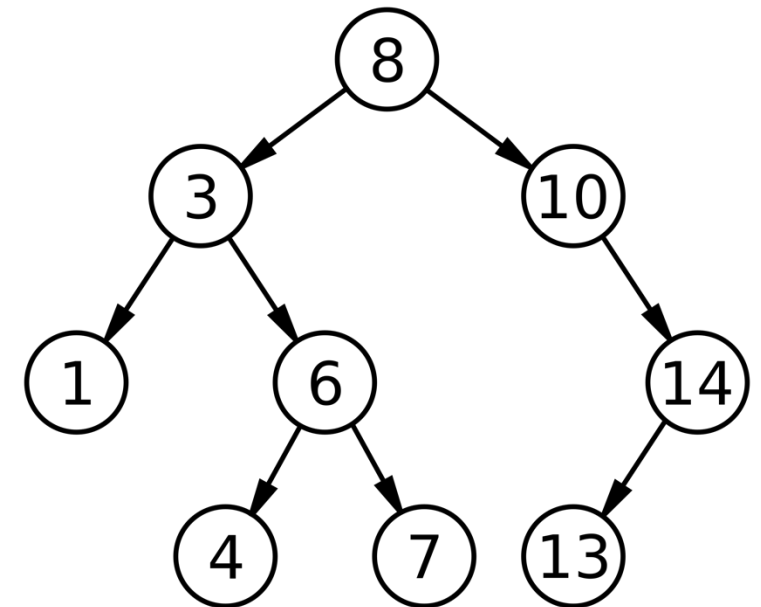
Arboles binarios de búsqueda (ABB | BST)

Sea A un árbol binario de raíz R e hijos izquierdo y derecho (posiblemente nulos) HI y HD , respectivamente.

Decimos que A es un árbol binario de búsqueda (ABB) si y solo si se satisfacen las dos condiciones al mismo tiempo:

- "HI es vacío" $\vee$ ("R es mayor que todo elemento de HI" $\wedge$ "HI es un ABB").
- "HD es vacío" $\vee$ ("R es menor que todo elemento de HD" $\wedge$ "HD es un ABB").

Donde " $\wedge$ " es la conjunción lógica "y", y " $\vee$ " es la disyunción lógica "o".

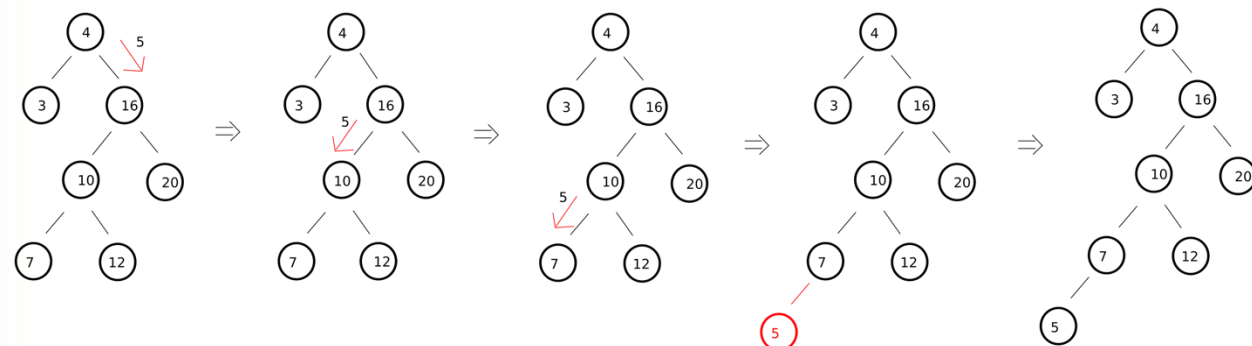


Un árbol binario de búsqueda de tamaño 9 y profundidad 3, con raíz 8 y hojas 1, 4, 7 y 13



UEA
UNIVERSIDAD
ESTATAL AMAZÓNICA

Introducción a los árboles binarios

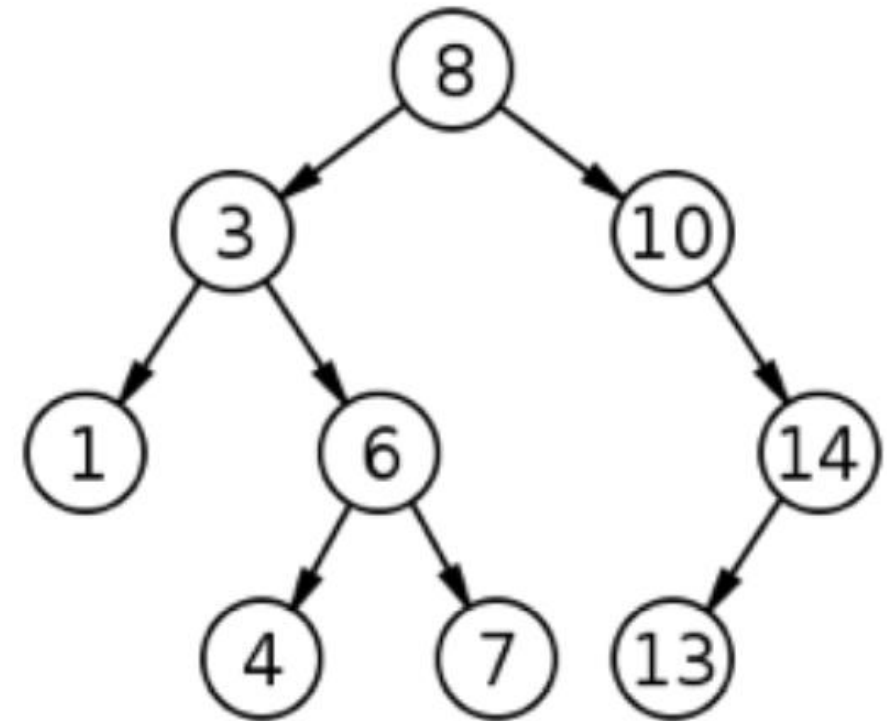




Árboles binarios de búsqueda

Un árbol binario de búsqueda (Binary Search Tree [BST]) es un árbol binario definido de la siguiente forma:

- Todo árbol vacío es un árbol binario de búsqueda.
- Un árbol binario no vacío, de raíz R, es un árbol binario de búsqueda si:
 - En caso de tener subárbol izquierdo, la raíz R debe ser mayor que el valor máximo almacenado en el subárbol izquierdo, y que el subárbol izquierdo sea un árbol binario de búsqueda.
 - En caso de tener subárbol derecho, la raíz R debe ser menor que el valor mínimo almacenado en el subárbol derecho, y que el subárbol derecho sea un árbol binario de búsqueda.





Arboles binarios de búsqueda – (key, value)

El **valor** es simplemente algo que se adjunta al nodo. Es la carga útil del nodo, por así decirlo. Es un **tipo primitivo** o un objeto arbitrario.

La **clave** se utiliza para determinar dónde insertar el nodo en el árbol. Tenga en cuenta que el valor no participa en absoluto en el desarrollo y mantenimiento de la estructura del árbol, como lo hace la clave.

Una buena analogía: un nodo es un contenedor. El **valor** es lo que se coloca en ese contenedor y la **clave** es cómo se marca el contenedor para poder recuperarlo más tarde.

```
void insert(int value)
{
    if(root == NULL)
        root = new Node(value);
    else
        insertHelper(root, value);
}

void insertHelper(Node* node, int value)
{
    if(value < node->key)
    {
        if(node->leftChild == NULL)
            node->leftChild = new Node(value);
        else
            insertHelper(node->leftChild, value);
    }
    else
    {
        if(node->rightChild == NULL)
            node->rightChild = new Node(value);
        else
            insertHelper(node->rightChild, value);
    }
}
```

Fuente: <https://stackoverflow.com/questions/16174999/binary-search-tree-what-is-the-difference-between-value-and-key?newreg=65d379a3fa6b435e99337ef3856c07f0>



Arboles binarios – Búsqueda

```
String getString(int llave) {  
    return buscar(llave, raiz);  
}  
  
String buscar(int llave, Nodo nodo) {  
    if (nodo==null)  
        fatalError("La llave "+llave+" no existe en el arreglo");  
    else if (llave==nodo.llave)  
        return nodo.val;  
    else if (llave<nodo.llave)  
        return buscar(llave, nodo.izq);  
    else  
        return buscar(llave, nodo.der);  
    return null; // Solo para evitar el error en compilacion  
}
```

Para implementar la operación getString se utiliza un patrón de programación denominado búsqueda en ABBs. Esta búsqueda hace uso de un método recursivo que llamaremos buscar:

Fuente: <https://users.dcc.uchile.cl/~lmateu/CC10A/Apuntes/arboles/#:~:text=Un%20%C3%A1rbol%20binario%20es%20un,en%20cada%20nodo%20una%20llave.>



Arboles binarios de búsqueda – Búsqueda recursiva

¿Cómo funciona la búsqueda recursiva?

La idea es comparar el valor que buscamos con el valor del nodo actual. Si son iguales, hemos encontrado el elemento. Si el valor buscado es menor, continuamos la búsqueda en el subárbol izquierdo. Si es mayor, continuamos en el subárbol derecho. Este proceso se repite recursivamente hasta encontrar el elemento o llegar a un nodo nulo.

Ventajas de la búsqueda recursiva:

- **Intuitiva:** La lógica es similar a la forma en que buscaríamos un valor en un diccionario ordenado.
- **Eficiente:** En un ABB balanceado, la búsqueda tiene una complejidad temporal promedio de $O(\log n)$.
- **Fácil de implementar:** La recursividad permite una solución elegante y concisa.

```
public class ArbolBinarioBusqueda
{
    private Nodo raiz;

    public bool Buscar(int valor)
    {
        return BuscarRecursivo(raiz, valor);
    }

    private bool BuscarRecursivo(Nodo nodo, int valor)
    {
        if (nodo == null)
        {
            return false; // No se encontró
        }

        if (valor == nodo.Valor)
        {
            return true; // Se encontró
        }
        else if (valor < nodo.Valor)
        {
            return BuscarRecursivo(nodo.Izquierdo, valor);
        }
        else
        {
            return BuscarRecursivo(nodo.Derecho, valor);
        }
    }
}
```



Arboles binarios de búsqueda – Búsqueda recursiva

```
// Creamos un BST
```

```
ArbolBinarioBusqueda arbol = new ArbolBinarioBusqueda();
```

```
// ... (Insertar nodos en el árbol)
```

```
// Buscar un valor
```

```
bool encontrado = arbol.Buscar(10);
```

```
if (encontrado)
```

```
{
```

```
    Console.WriteLine("El valor se encontró en el árbol");
```

```
}
```

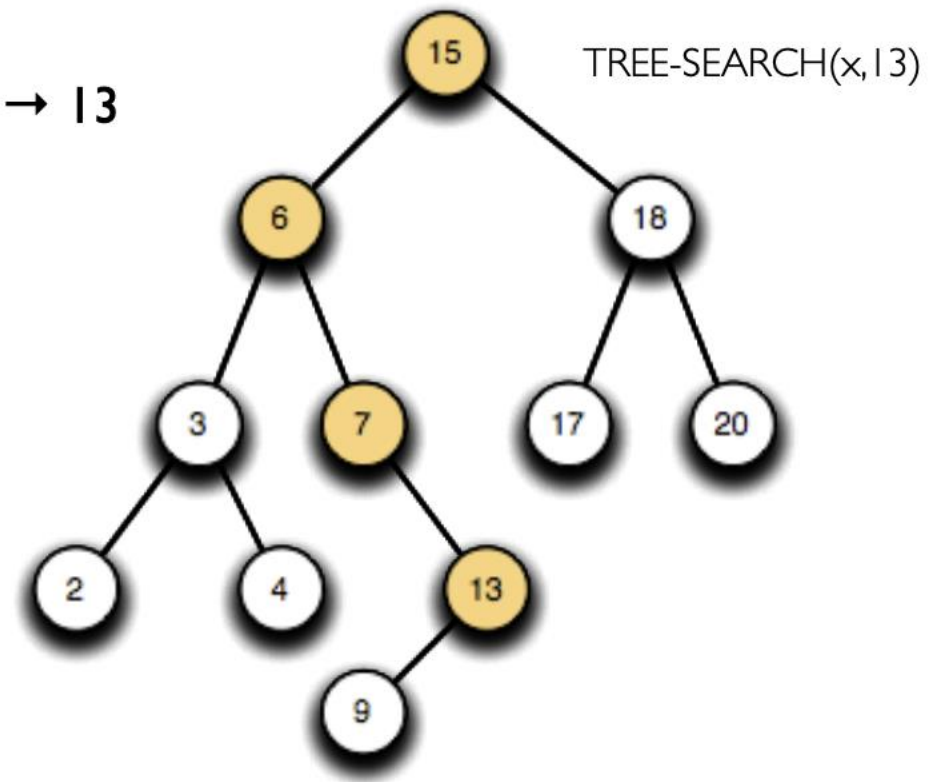
```
else
```

```
{
```

```
    Console.WriteLine("El valor no se encontró en el árbol");
```

```
}
```

15 → 6 → 7 → 13





Arboles binarios de búsqueda – Búsqueda iterativa

La búsqueda iterativa en un BST es un enfoque alternativo a la recursividad para encontrar un valor dentro del árbol. En lugar de utilizar llamadas a funciones anidadas, se emplea un bucle para recorrer el árbol de nodo en nodo hasta encontrar el valor o llegar a un nodo nulo.

Cómo funciona:

1. Inicio en la raíz: Se inicia la búsqueda en el nodo raíz del árbol.
2. Comparación: Se compara el valor buscado con el valor del nodo actual.
3. Descenso:
 - Si el valor buscado es menor que el valor del nodo actual, se mueve al subárbol izquierdo.
 - Si el valor buscado es mayor que el valor del nodo actual, se mueve al subárbol derecho.
4. Repetición: Los pasos 2 y 3 se repiten hasta encontrar el valor o llegar a un nodo nulo.

Ventajas:

- **Mayor control:** Se puede realizar un seguimiento más detallado del proceso de búsqueda.
- **Potencialmente más eficiente:** En algunos casos, la iteración puede ser más eficiente que la recursión debido a la sobrecarga de llamadas a funciones.
- **Evita desbordamiento de pila:** Al no utilizar llamadas recursivas, se reduce el riesgo de desbordamiento de la pila, especialmente en árboles muy grandes.

```
public class ArbolBinarioBusqueda
{
    private Nodo raiz;

    public bool Buscar(int valor)
    {
        Nodo actual = raiz;

        while (actual != null)
        {
            if (valor == actual.Valor)
            {
                return true; // Valor encontrado
            }
            else if (valor < actual.Valor)
            {
                actual = actual.Izquierdo;
            }
            else
            {
                actual = actual.Derecho;
            }
        }

        return false; // Valor no encontrado
    }
}
```

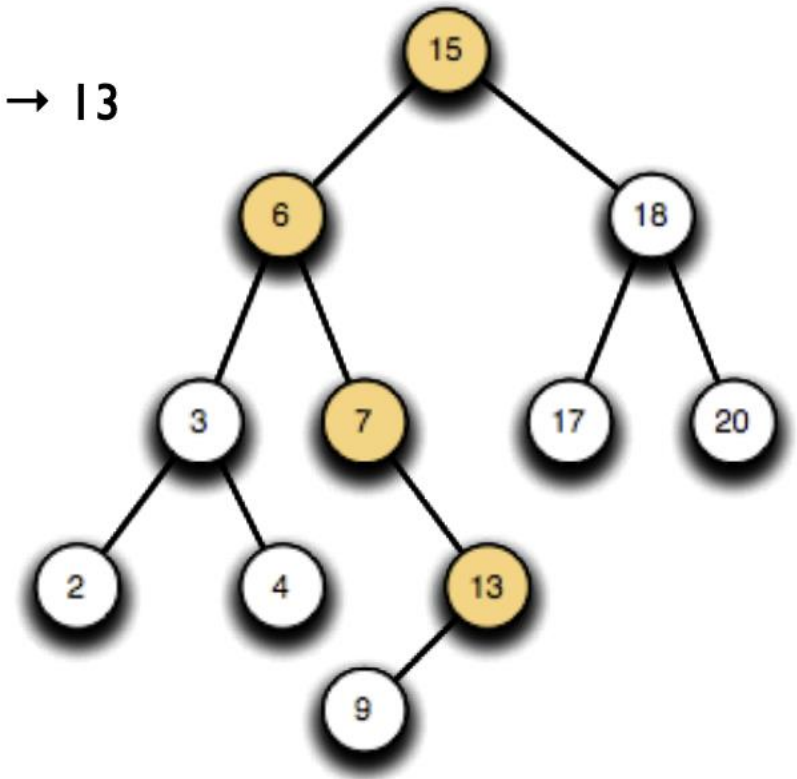


Arboles binarios de búsqueda – Búsqueda iterativa

```
// Creamos un BST
ArbolBinarioBusqueda arbol = new ArbolBinarioBusqueda();
// ... (Insertar nodos en el árbol)

// Buscar un valor
bool encontrado = arbol.Buscar(10);
if (encontrado)
{
    Console.WriteLine("El valor se encontró en el árbol");
}
else
{
    Console.WriteLine("El valor no se encontró en el árbol");
}
```

15 → 6 → 7 → 13





Arboles binarios de búsqueda – Elemento mínimo

En un Árbol Binario de Búsqueda (BST), el elemento mínimo siempre se encuentra en el nodo más a la izquierda del árbol. Esto se debe a la propiedad fundamental de los BST: todos los nodos del subárbol izquierdo de un nodo tienen valores menores que el valor del nodo.

```
public class ArbolBinarioBusqueda
{
    private Nodo raiz;

    public int EncontrarMinimo()
    {
        Nodo actual = raiz;

        while (actual.Izquierdo != null)
        {
            actual = actual.Izquierdo;
        }

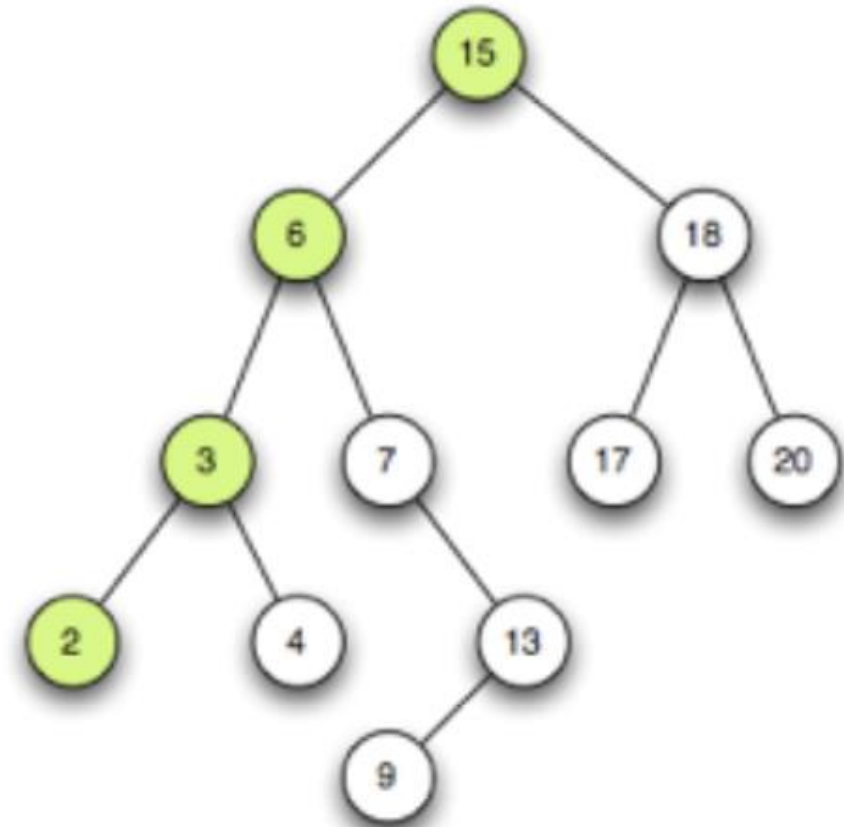
        return actual.Valor;
    }
}
```



Arboles binarios de búsqueda – Elemento mínimo

```
// Creamos un BST
ArbolBinarioBusqueda arbol = new ArbolBinarioBusqueda();
// ... (Insertar nodos en el árbol)

// Encontrar el mínimo
int minimo = arbol.EncontrarMinimo();
Console.WriteLine("El valor mínimo es: " + minimo);
```





Arboles binarios de búsqueda – Elemento máximo

En un Árbol Binario de Búsqueda (BST), el elemento máximo siempre se encuentra en el nodo más a la derecha del árbol. Esto se debe a que todos los nodos del subárbol derecho de un nodo tienen valores mayores que el valor del nodo.

```
public class ArbolBinarioBusqueda
{
    private Nodo raiz;

    public int EncontrarMaximo()
    {
        Nodo actual = raiz;

        while (actual.Derecho != null)
        {
            actual = actual.Derecho;
        }

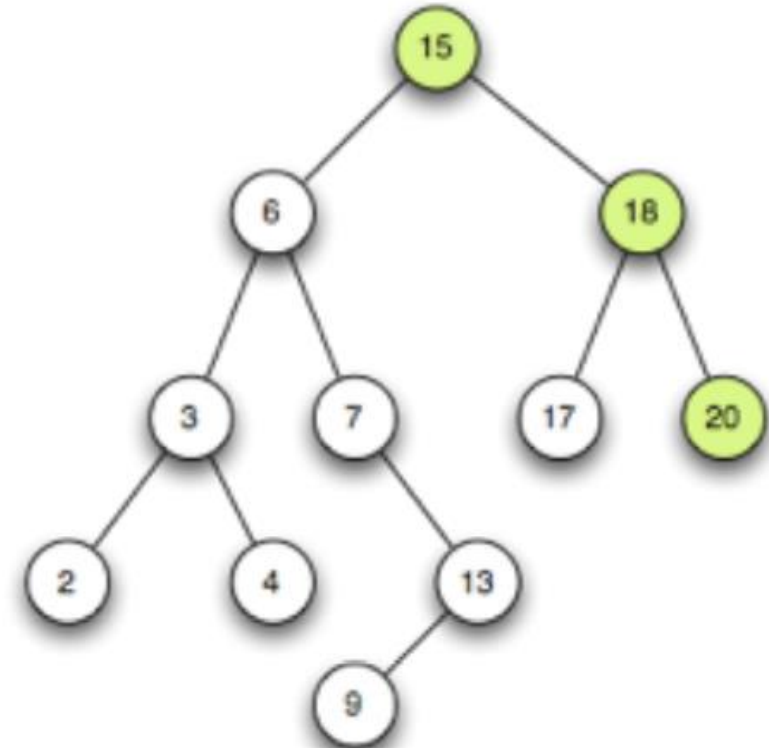
        return actual.Valor;
    }
}
```



Arboles binarios de búsqueda – Elemento máximo

```
// Creamos un BST
ArbolBinarioBusqueda arbol = new ArbolBinarioBusqueda();
// ... (Insertar nodos en el árbol)

// Encontrar el máximo
int maximo = arbol.EncontrarMaximo();
Console.WriteLine("El valor máximo es: " + maximo);
```





UEA
UNIVERSIDAD
ESTATAL AMAZÓNICA

Gracias



UEA
UNIVERSIDAD
ESTATAL AMAZÓNICA